Understanding the SIREN Frequency Parameter ω_0: Architecture, Training, Initialization, and Higher-Order Derivatives

Internal technical note

July 9, 2026

Contents

1 Purpose of this note

This note explains the role of the SIREN frequency-scale parameter, usually written as ω_0, in the context of a coordinate-based implicit neural representation for radar plasma-density reconstruction.

The specific practical problem is the following. We are using a SIREN-style neural network to approximate a continuous field

$$f_\theta(\tilde{x}, \tilde{y}, \tilde{t}) \approx \widetilde{\log_{10} N_e}, \tag{1}$$

where $\tilde{x}$, $\tilde{y}$, and $\tilde{t}$ are normalized spatial and temporal coordinates, and the output is a normalized version of $\log_{10} N_e$.

The central questions are:

(1) What is the difference between the trainable weights W_ℓ and the SIREN frequency-scale hyperparameter ω_0?

(2) Why does the SIREN equation appear to contain two different "frequency-like" objects?

(3) If ω_0 can be absorbed into the trainable weights, why is ω_0 not normally trained?

(4) What makes SIREN different from simply using sine as an activation function?

(5) How does ω_0 affect gradients, Hessians, and higher-order derivatives?

(6) Is the current radar implementation aligned with the original SIREN idea?

The short answer is:

$$\boxed{\omega_0 \text{ is not a learned physical frequency.}} \tag{2}$$

It is better understood as a fixed activation-scale hyperparameter that controls the initial spectral regime, the conditioning of the optimization, and the derivative scales of the network. The trainable weights W_ℓ and biases b_ℓ learn the function. The fixed ω_0 controls how the sine nonlinearity is exposed to those learned affine transformations.

2 Acronyms and terminology

3 Coordinate-based implicit neural representation

A coordinate-based INR represents a field as a neural network that maps coordinates directly to values. Instead of storing a value on a fixed grid, the field is represented continuously:

$$f_\theta : \mathbb{R}^d \to \mathbb{R}^m. \tag{3}$$

For the current radar application,

$$d = 3, \qquad m = 1, \tag{4}$$

and the input coordinate is

Term	Meaning
INR	Implicit Neural Representation
SIREN	Sinusoidal Representation Network
MLP	Multi-Layer Perceptron
AMISR	Advanced Modular Incoherent Scatter Radar
ISR	Incoherent Scatter Radar
MSE	Mean Squared Error
PDE	Partial Differential Equation
BVP	Boundary Value Problem
AD	Automatic Differentiation
ADAM	Adaptive Moment Estimation optimizer
Hessian	Matrix of second partial derivatives
Laplacian	Trace of the Hessian

$$\boldsymbol{r} = \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{t} \end{bmatrix} \in \mathbb{R}^3. \tag{5}$$

The target is a normalized electron-density value:

$$\tilde{u} = \widetilde{\log_{10} N_e}. \tag{6}$$

The model therefore represents

$$\tilde{u} = f_\theta(\tilde{x}, \tilde{y}, \tilde{t}). \tag{7}$$

After training, predictions are denormalized back to physical units:

$$u = \log_{10} N_e. \tag{8}$$

4 Coordinate normalization

The code normalizes each coordinate to the interval $[-1, 1]$. For example,

$$\tilde{x} = 2\frac{x - x_{\min}}{x_{\max} - x_{\min}} - 1, \tag{9}$$

$$\tilde{y} = 2\frac{y - y_{\min}}{y_{\max} - y_{\min}} - 1, \tag{10}$$

$$\tilde{t} = 2\frac{t - t_{\min}}{t_{\max} - t_{\min}} - 1. \tag{11}$$

The target may also be normalized:

$$\tilde{u} = 2\frac{u - u_{\min}}{u_{\max} - u_{\min}} - 1. \tag{12}$$

This detail is essential for interpreting ω_0. The number $\omega_0 = 5$ does not mean "5 radians per kilometer" or "5 cycles per hour". It acts on normalized coordinates. Therefore, its physical meaning depends on the coordinate scaling.

For example, since

$$\tilde{x} = 2\frac{x - x_{\min}}{x_{\max} - x_{\min}} - 1, \tag{13}$$

we have

$$\frac{\partial \tilde{x}}{\partial x} = \frac{2}{x_{\max} - x_{\min}}. \tag{14}$$

Define

$$s_x = \frac{2}{x_{\max} - x_{\min}}, \qquad s_y = \frac{2}{y_{\max} - y_{\min}}, \qquad s_t = \frac{2}{t_{\max} - t_{\min}}. \tag{15}$$

Then derivatives with respect to normalized coordinates and derivatives with respect to physical coordinates are related by chain-rule scale factors.

For a model output $\tilde{u} = f_\theta(\tilde{x}, \tilde{y}, \tilde{t})$, the first physical derivative is

$$\frac{\partial \tilde{u}}{\partial x} = \frac{\partial \tilde{u}}{\partial \tilde{x}}\frac{\partial \tilde{x}}{\partial x} = s_x \frac{\partial \tilde{u}}{\partial \tilde{x}}. \tag{16}$$

The second physical derivative is

$$\frac{\partial^2 \tilde{u}}{\partial x^2} = s_x^2 \frac{\partial^2 \tilde{u}}{\partial \tilde{x}^2}. \tag{17}$$

Similarly,

$$\frac{\partial^2 \tilde{u}}{\partial x \partial y} = s_x s_y \frac{\partial^2 \tilde{u}}{\partial \tilde{x} \partial \tilde{y}}, \tag{18}$$

and

$$\frac{\partial^2 \tilde{u}}{\partial t^2} = s_t^2 \frac{\partial^2 \tilde{u}}{\partial \tilde{t}^2}. \tag{19}$$

If one also wants derivatives of the denormalized output

$$u = \frac{u_{\max} - u_{\min}}{2}(\tilde{u} + 1) + u_{\min}, \tag{20}$$

then the physical derivative contains the target scale factor

$$s_u = \frac{u_{\max} - u_{\min}}{2}. \tag{21}$$

Thus,

$$\frac{\partial^2 u}{\partial x^2} = s_u s_x^2 \frac{\partial^2 \tilde{u}}{\partial \tilde{x}^2}. \tag{22}$$

This means that the curvature losses currently computed in normalized coordinates are mathematically valid, but they are dimensionless normalized-coordinate curvature penalties unless physical rescaling is explicitly included.

5 Original SIREN layer

The original SIREN paper defines a sine-based coordinate MLP. A generic hidden layer is written as an affine map followed by a sine activation:

$$\phi_i(\boldsymbol{x}_i) = \sin(W_i \boldsymbol{x}_i + \boldsymbol{b}_i). \tag{23}$$

The full network is a composition of such layers followed by a final linear map:

$$\Phi(\boldsymbol{x}) = W_n \left(\phi_{n-1} \circ \phi_{n-2} \circ \cdots \circ \phi_0\right)(\boldsymbol{x}) + \boldsymbol{b}_n. \tag{24}$$

In a practical implementation, especially in the official code and in the radar code, the sine activation is often written with an explicit scale parameter:

$$h_\ell = \sin\left(\omega_\ell \left(W_\ell h_{\ell-1} + \boldsymbol{b}_\ell\right)\right). \tag{25}$$

This is the equation that caused confusion. It appears to contain both ω_ℓ and W_ℓ, and both seem to influence frequency.

That observation is correct.

The distinction is:

$$\boxed{W_\ell, \boldsymbol{b}_\ell \text{ are trainable neural-network parameters.}} \tag{26}$$

$$\boxed{\omega_\ell \text{ is usually a fixed hyperparameter.}} \tag{27}$$

The trainable parameter set is

$$\theta = \{W_\ell, \boldsymbol{b}_\ell\}_{\ell=1}^{L}. \tag{28}$$

In the current implementation, ω_ℓ is not part of θ.

6 The two "w" issue: W_ℓ versus ω_0

6.1 Why the notation is confusing

A SIREN layer may be written as

$$h_\ell = \sin\left(\omega_\ell (W_\ell h_{\ell-1} + \boldsymbol{b}_\ell)\right). \tag{29}$$

Here:

- W_ℓ is a matrix of trainable weights.
- $\boldsymbol{b}_\ell$ is a vector of trainable biases.
- ω_ℓ is a fixed scalar frequency-scale factor.

Because W_ℓ can also be interpreted as controlling frequency, the use of ω_ℓ creates a legitimate conceptual question:

$$\text{Why do we need both } \omega_\ell \text{ and } W_\ell? \tag{30}$$

The mathematical reason is not expressivity. The reason is initialization and optimization.

6.2 Absorbing ω_ℓ into the weights

For a single layer,

$$h_\ell = \sin\left(\omega_\ell(W_\ell h_{\ell-1} + \boldsymbol{b}_\ell)\right). \tag{31}$$

This can be rewritten as

$$h_\ell = \sin\left(\tilde{W}_\ell h_{\ell-1} + \tilde{\boldsymbol{b}}_\ell\right), \tag{32}$$

where

$$\tilde{W}_\ell = \omega_\ell W_\ell, \tag{33}$$

and

$$\tilde{\boldsymbol{b}}_\ell = \omega_\ell \boldsymbol{b}_\ell. \tag{34}$$

Therefore, as a pure function class,

$$\omega_\ell \text{ is redundant.} \tag{35}$$

A network with $\omega_\ell = 1$ can represent the same layer by learning weights that are ω_ℓ times larger.

For example,

$$\sin(5wx + b) \tag{36}$$

can be represented as

$$\sin\Big(\tilde{w}x + \tilde{b}\Big), \tag{37}$$

with

$$\tilde{w} = 5w, \qquad \tilde{b} = 5b. \tag{38}$$

This means the correct argument for using $\omega_0 = 5$ is not:

$$\omega_0 = 5 \text{ gives the network functions that } \omega_0 = 1 \text{ cannot represent.} \tag{39}$$

That statement is false in the broad expressivity sense.

The correct statement is:

$$\boxed{\omega_0 \text{ changes the initialization, optimization trajectory, derivative scales, and inductive bias.}} \tag{40}$$

7 Why ω_0 is not normally trained

7.1 Concrete code-level reason

In the radar implementation, the sine activation is defined conceptually as

```
class Sine(nn.Module):
    def __init__(self, w0=30.0):
        super().__init__()
        self.w0 = float(w0)

    def forward(self, x):
        return torch.sin(self.w0 * x)
```

The important line is

```
self.w0 = float(w0)
```

Since `self.w0` is a Python float and not a `torch.nn.Parameter`, it is not included in

```
model.parameters()
```

Therefore, when the optimizer is created using

```
optimizer = torch.optim.Adam(model.parameters(), lr=args.lr)
```

the optimizer updates the linear-layer weights and biases, but not ω_0.

Thus, in the current code:

$$\boxed{\omega_0 \text{ is fixed because it is not registered as a trainable parameter.}} \tag{41}$$

7.2 Conceptual reason: non-identifiability

Even if we made ω_0 trainable, it would be non-identifiable because it is multiplicatively redundant with W_ℓ.

For a neuron

$$h(x) = \sin(\omega_0(wx + b)), \tag{42}$$

the following parameter settings can represent the same function:

$$(\omega_0, w, b) = (5, w, b), \tag{43}$$

$$(\omega_0, w, b) = (1, 5w, 5b), \tag{44}$$

$$(\omega_0, w, b) = (10, 0.5w, 0.5b). \tag{45}$$

All of these produce the same argument to the sine:

$$\omega_0(wx + b). \tag{46}$$

Therefore, ω_0 is not a uniquely recoverable physical quantity. It is not like a measured plasma frequency. It is a parameterization scale.

If ω_0 and W_ℓ are both trainable, the optimizer has an extra redundant direction in parameter space. This may make training less stable, not more meaningful.

7.3 Conceptual reason: the "best" value depends on the objective

In a sparse radar reconstruction problem, many continuous functions can pass through the measured radar points. The lowest training MSE is not necessarily the most physically plausible plasma field.

If ω_0 were trainable, and if the data loss dominated the objective, the optimizer could increase effective frequency content to fit individual measured points more aggressively. This could reduce

$$L_{\text{data}} \tag{47}$$

while creating unphysical oscillations between radar beams.

The scientific objective is closer to

$$\text{find a plausible smooth plasma field compatible with the measurements,} \tag{48}$$

not merely

$$\text{minimize pointwise error at sparse radar samples.} \tag{49}$$

Therefore, using a fixed low value such as

$$\omega_0 = 5 \tag{50}$$

is a modeling prior. It biases the network toward smoother, larger-scale structure.

7.4 Can ω_0 be trained?

Yes. Nothing mathematically prevents it.

One could define

```
self.log_w0 = nn.Parameter(torch.tensor(np.log(5.0)))
```

and then use

```
w0 = torch.exp(self.log_w0)
return torch.sin(w0 * x)
```

This would make ω_0 positive and trainable.

However, doing so changes the interpretation. The model would no longer use ω_0 purely as a fixed spectral prior. It would become a trainable scale factor, redundant with the weights and influenced by the loss balance.

If trainable ω_0 is tested, it should be treated as an ablation, not as an automatically superior model.

A useful ablation set would be:

$$\omega_0 = 1 \text{ fixed}, \tag{51}$$

$$\omega_0 = 5 \text{ fixed}, \tag{52}$$

$$\omega_0 = 30 \text{ fixed}, \tag{53}$$

$$\omega_0 \text{ trainable, initialized at } 1, \tag{54}$$

$$\omega_0 \text{ trainable, initialized at } 5. \tag{55}$$

The comparison should not use only training MSE. It should also examine:

- measured-point RMSE,
- curvature magnitude,
- visual morphology between beams,
- overlap disagreement between temporal windows,
- stability across random seeds,
- behavior under different regularization strengths.

8 What makes SIREN different from simply using sine as an activation?

A naive sine-activation MLP would be

$$h_\ell = \sin(W_\ell h_{\ell-1} + \boldsymbol{b}_\ell), \tag{56}$$

with a standard initialization such as Xavier or Kaiming initialization.
SIREN is more specific. It is not merely "use sine". It is the combination of:

(1) coordinate-based input,

(2) sinusoidal hidden activations,

(3) a linear output layer,

(4) special first-layer initialization,

(5) special hidden-layer initialization,

(6) an activation scale ω_0,

(7) the use of automatic differentiation to compute derivatives with respect to input coordinates.

The original SIREN paper emphasizes that initialization is necessary for stable training. The reason is that the sine function is periodic and non-monotonic. If the pre-activation scale is too small, the network behaves almost linearly. If the pre-activation scale is too large, the network may become highly oscillatory and hard to optimize.

The SIREN initialization is designed so that activation statistics remain controlled through depth.

9 SIREN initialization

9.1 First layer

For the first layer, the current code uses

$$W_1 \sim \mathcal{U}\left(-\frac{1}{n_{\text{in}}}, \frac{1}{n_{\text{in}}}\right), \tag{57}$$

where n_{in} is the number of input coordinates.

Then the activation is

$$h_1 = \sin\left(\omega_{\text{first}}(W_1 \boldsymbol{r} + \boldsymbol{b}_1)\right). \tag{58}$$

The first layer is special because it sees the actual coordinates. Therefore, ω_{first} directly controls the initial coordinate-domain oscillation scale.

For the current radar model,

$$\boldsymbol{r} = \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{t} \end{bmatrix}. \tag{59}$$

Thus,

$$h_1 = \sin\left(\omega_{\text{first}}(W_1[\tilde{x}, \tilde{y}, \tilde{t}]^T + \boldsymbol{b}_1)\right). \tag{60}$$

Choosing

$$\omega_{\text{first}} = 5 \tag{61}$$

instead of

$$\omega_{\text{first}} = 30 \tag{62}$$

means the first layer starts with lower coordinate-domain frequency content.

9.2 Hidden layers

For hidden layers, the current code uses

$$W_\ell \sim \mathcal{U}\left(-\frac{\sqrt{6/n_{\text{in}}}}{\omega_{\text{hidden}}}, \frac{\sqrt{6/n_{\text{in}}}}{\omega_{\text{hidden}}}\right). \tag{63}$$

The corresponding hidden activation is

$$h_\ell = \sin\left(\omega_{\text{hidden}}(W_\ell h_{\ell-1} + \boldsymbol{b}_\ell)\right). \tag{64}$$

The division by ω_{hidden} in the initialization is important. It compensates for the multiplication by ω_{hidden} inside the sine activation.

At initialization, the effective hidden-layer pre-sine scale is approximately

$$\omega_{\text{hidden}} W_\ell \sim \omega_{\text{hidden}} \cdot \frac{\sqrt{6/n_{\text{in}}}}{\omega_{\text{hidden}}} = \sqrt{\frac{6}{n_{\text{in}}}}. \tag{65}$$

Therefore, changing ω_{hidden} does not automatically explode the hidden-layer pre-activation scale at initialization, because the weight initialization compensates for it.

The first layer does not have this same compensation in the same way. That is why ω_{first} is especially important as a coordinate-domain spectral knob.

10 Forward pass in the radar SIREN

The current radar model has the form

$$f_\theta : \mathbb{R}^3 \to \mathbb{R}. \tag{66}$$

Let

$$h_0 = \boldsymbol{r} = \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{t} \end{bmatrix}. \tag{67}$$

The first layer is

$$z_1 = W_1 h_0 + \boldsymbol{b}_1, \tag{68}$$

$$h_1 = \sin(\omega_{\text{first}} z_1). \tag{69}$$

The hidden layers are

$$z_\ell = W_\ell h_{\ell-1} + \boldsymbol{b}_\ell, \tag{70}$$

$$h_\ell = \sin(\omega_{\text{hidden}} z_\ell), \qquad \ell = 2, \ldots, L. \tag{71}$$

The output layer is linear:

$$\hat{u} = W_{\text{out}} h_L + \boldsymbol{b}_{\text{out}}. \tag{72}$$

Thus,

$$\boxed{f_\theta(\boldsymbol{r}) = W_{\text{out}} h_L + \boldsymbol{b}_{\text{out}}.} \tag{73}$$

There is no final sine activation. This is appropriate because the output is a scalar regression target.

11 Training objective

The basic measured-data loss is

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta(\boldsymbol{r}_i) - \tilde{u}_i \right)^2. \tag{74}$$

The current radar training objective adds curvature regularization:

$$L = L_{\text{data}} + \lambda_{xy} L_{xy} + \lambda_t L_t. \tag{75}$$

The spatial curvature loss is

$$L_{xy} = \mathbb{E}_{\boldsymbol{r}_c}\left[f_{\tilde{x}\tilde{x}}^2 + 2f_{\tilde{x}\tilde{y}}^2 + f_{\tilde{y}\tilde{y}}^2\right], \tag{76}$$

where the derivatives are with respect to normalized coordinates.

The temporal curvature loss is

$$L_t = \mathbb{E}_{\boldsymbol{r}_c}\left[f_{\tilde{t}\tilde{t}}^2\right]. \tag{77}$$

The use of second derivatives is important. It penalizes rapid bending but does not penalize a linear trend.

For example, in one dimension,

$$f(x) = ax + b \tag{78}$$

has

$$f_x = a, \tag{79}$$

but

$$f_{xx} = 0. \tag{80}$$

Therefore, curvature regularization allows large-scale gradients while discouraging unnecessary oscillation.

12 Why SIREN is useful for derivative losses

SIREN is especially useful for derivative-based losses because the derivative of sine is another sinusoidal function:

$$\frac{d}{dz}\sin(z) = \cos(z), \tag{81}$$

and

$$\cos(z) = \sin\left(z + \frac{\pi}{2}\right). \tag{82}$$

Thus, the derivative of a sine is a phase-shifted sine. Higher derivatives continue to be sinusoidal:

$$\frac{d^2}{dz^2}\sin(z) = -\sin(z), \tag{83}$$

$$\frac{d^3}{dz^3}\sin(z) = -\cos(z), \tag{84}$$

$$\frac{d^4}{dz^4}\sin(z) = \sin(z). \tag{85}$$

This periodic derivative structure is fundamentally different from ReLU. A ReLU network is piecewise linear, so its second derivative is zero almost everywhere. That is not ideal when the loss explicitly uses second derivatives.

13 Propagation of ω_0 through derivatives: one-dimensional single-neuron case

Consider the simplest one-dimensional SIREN neuron:

$$f(x) = \sin\left(\omega_0(wx + b)\right). \tag{86}$$

Define

$$a = \omega_0 w, \tag{87}$$

and

$$c = \omega_0 b. \tag{88}$$

Then

$$f(x) = \sin(ax + c). \tag{89}$$

The first derivative is

$$f_x(x) = a\cos(ax + c). \tag{90}$$

Substituting back,

$$f_x(x) = \omega_0 w \cos\left(\omega_0(wx + b)\right). \tag{91}$$

The second derivative is

$$f_{xx}(x) = -a^2 \sin(ax + c). \tag{92}$$

Substituting back,

$$f_{xx}(x) = -(\omega_0 w)^2 \sin\left(\omega_0(wx + b)\right). \tag{93}$$

The third derivative is

$$f_{xxx}(x) = -a^3 \cos(ax + c), \tag{94}$$

or

$$f_{xxx}(x) = -(\omega_0 w)^3 \cos\left(\omega_0(wx + b)\right). \tag{95}$$

The fourth derivative is

$$f_{xxxx}(x) = a^4 \sin(ax + c), \tag{96}$$

or

$$f_{xxxx}(x) = (\omega_0 w)^4 \sin\left(\omega_0(wx + b)\right). \tag{97}$$

In general, the k-th derivative scales like

$$\frac{d^k f}{dx^k} \sim (\omega_0 w)^k. \tag{98}$$

This is the first key point:

$$\boxed{\omega_0 \text{ enters the } k\text{-th derivative approximately as } \omega_0^k.} \tag{99}$$

Therefore, ω_0 affects not only the apparent frequency of the function but also the magnitude of gradients, Hessians, and higher-order derivatives.

For curvature regularization, this matters strongly because curvature involves second derivatives. In the single-neuron one-dimensional case,

$$f_{xx}^2 = (\omega_0 w)^4 \sin^2\left(\omega_0(wx+b)\right). \tag{100}$$

Thus, a second-derivative squared loss has a fourth-power dependence on the effective frequency scale:

$$f_{xx}^2 \sim \omega_0^4 w^4. \tag{101}$$

This is why ω_0 is not a harmless cosmetic parameter when derivative losses are used.

14 Propagation through derivatives: multidimensional single-neuron case

Now consider a scalar neuron with multidimensional input:

$$f(\boldsymbol{x}) = \sin\left(\omega_0(\boldsymbol{w}^T\boldsymbol{x}+b)\right). \tag{102}$$

Let

$$q(\boldsymbol{x}) = \omega_0(\boldsymbol{w}^T\boldsymbol{x}+b). \tag{103}$$

Then

$$f(\boldsymbol{x}) = \sin(q(\boldsymbol{x})). \tag{104}$$

The gradient is

$$\nabla_{\boldsymbol{x}} f = \cos(q)\nabla_{\boldsymbol{x}} q. \tag{105}$$

Since

$$\nabla_{\boldsymbol{x}} q = \omega_0 \boldsymbol{w}, \tag{106}$$

we get

$$\nabla_{\boldsymbol{x}} f = \omega_0 \cos(q)\boldsymbol{w}. \tag{107}$$

The Hessian is

$$\nabla_{\boldsymbol{x}}^2 f = -\sin(q)\left(\nabla_{\boldsymbol{x}} q\right)\left(\nabla_{\boldsymbol{x}} q\right)^T. \tag{108}$$

Therefore,

$$\nabla_{\boldsymbol{x}}^2 f = -\omega_0^2 \sin(q)\boldsymbol{w}\boldsymbol{w}^T. \tag{109}$$

The individual second derivatives are

$$\frac{\partial^2 f}{\partial x_i \partial x_j} = -\omega_0^2 w_i w_j \sin\left(\omega_0(\boldsymbol{w}^T \boldsymbol{x} + b)\right). \tag{110}$$

Thus,

$$f_{x_i x_j} \propto \omega_0^2 w_i w_j. \tag{111}$$

For the radar horizontal Hessian penalty,

$$f_{\tilde{x}\tilde{x}}^2 + 2f_{\tilde{x}\tilde{y}}^2 + f_{\tilde{y}\tilde{y}}^2, \tag{112}$$

each second derivative contains a factor ω_0^2, and each squared second derivative contains ω_0^4.

This means that changing ω_0 changes how easy it is for the model to create curvature, and how strongly curvature appears in the loss.

15 Propagation through a full multilayer SIREN

A full SIREN has several layers. Let

$$z_\ell = W_\ell h_{\ell-1} + \boldsymbol{b}_\ell, \tag{113}$$

and

$$h_\ell = \sin(\omega_\ell z_\ell). \tag{114}$$

Define the elementwise derivative matrix

$$D_\ell = \operatorname{diag}\left[\cos(\omega_\ell z_\ell)\right]. \tag{115}$$

The Jacobian of layer ℓ with respect to its input $h_{\ell-1}$ is

$$J_\ell = \frac{\partial h_\ell}{\partial h_{\ell-1}} = \omega_\ell D_\ell W_\ell. \tag{116}$$

For the full network, ignoring the final linear layer for a moment, the first derivative with respect to the input coordinates contains a product of these Jacobians:

$$\frac{\partial h_L}{\partial \boldsymbol{x}} = J_L J_{L-1} \cdots J_1. \tag{117}$$

Therefore,

$$\frac{\partial h_L}{\partial \boldsymbol{x}} = (\omega_L D_L W_L)(\omega_{L-1} D_{L-1} W_{L-1}) \cdots (\omega_1 D_1 W_1). \tag{118}$$

This shows that first derivatives can scale with products of effective layer gains:

$$\prod_{\ell=1}^{L} \omega_\ell \|W_\ell\|. \tag{119}$$

This is why SIREN initialization matters. If each $\omega_\ell W_\ell$ is too large, gradients can become very large or highly oscillatory. If each $\omega_\ell W_\ell$ is too small, the network can behave too linearly and may fail to represent detail.

15.1 Second derivatives in a multilayer SIREN

Second derivatives are more complicated because they include two kinds of terms:

(1) terms from differentiating the linear transformations through the first derivatives,

(2) terms from the second derivative of the sine activation.

For a layer

$$h_\ell = \sin(\omega_\ell z_\ell), \tag{120}$$

the first elementwise derivative is

$$\frac{dh_\ell}{dz_\ell} = \omega_\ell \cos(\omega_\ell z_\ell). \tag{121}$$

The second elementwise derivative is

$$\frac{d^2 h_\ell}{dz_\ell^2} = -\omega_\ell^2 \sin(\omega_\ell z_\ell). \tag{122}$$

Thus, every time the second derivative of the activation appears, it introduces a factor

$$\omega_\ell^2. \tag{123}$$

For a two-layer scalar example,

$$f(x) = a_2 \sin\left(\omega_2 \left[w_2 \sin\left(\omega_1(w_1 x + b_1)\right) + b_2\right]\right), \tag{124}$$

define

$$u(x) = \omega_1(w_1 x + b_1), \tag{125}$$

$$h_1(x) = \sin(u(x)), \tag{126}$$

$$v(x) = \omega_2(w_2 h_1(x) + b_2). \tag{127}$$

Then

$$f(x) = a_2 \sin(v(x)). \tag{128}$$

The first derivative is

$$f_x = a_2 \cos(v) v_x. \tag{129}$$

Now

$$v_x = \omega_2 w_2 h_{1,x}, \tag{130}$$

and

$$h_{1,x} = \omega_1 w_1 \cos(u). \tag{131}$$

Therefore,

$$f_x = a_2 \cos(v)\omega_2 w_2 \omega_1 w_1 \cos(u). \tag{132}$$

The first derivative already contains the product

$$\omega_1 \omega_2 w_1 w_2. \tag{133}$$

The second derivative is

$$f_{xx} = a_2 \left[-\sin(v)(v_x)^2 + \cos(v) v_{xx} \right]. \tag{134}$$

The first term contains

$$(v_x)^2 = \omega_2^2 w_2^2 \omega_1^2 w_1^2 \cos^2(u). \tag{135}$$

The second term contains

$$v_{xx} = \omega_2 w_2 h_{1,xx}. \tag{136}$$

But

$$h_{1,xx} = -\omega_1^2 w_1^2 \sin(u). \tag{137}$$

Thus,

$$v_{xx} = -\omega_2 w_2 \omega_1^2 w_1^2 \sin(u). \tag{138}$$

Putting this together,

$$\begin{aligned} f_{xx} = a_2 \Big[& -\sin(v)\omega_2^2 w_2^2 \omega_1^2 w_1^2 \cos^2(u) \\ & -\cos(v)\omega_2 w_2 \omega_1^2 w_1^2 \sin(u) \Big]. \end{aligned} \tag{139}$$

This expression shows two important facts:

(1) Higher-order derivatives contain powers and products of the ω_ℓ and W_ℓ factors.

(2) Even when ω_ℓ is fixed, it strongly affects derivative magnitudes and training gradients.

This explains why ω_0 matters especially in a model that uses curvature regularization.

16 Why low $\omega_0 = 5$ is defensible for sparse radar data

The original SIREN paper found $\omega_0 = 30$ useful across its applications. That does not imply $\omega_0 = 30$ is universally optimal.

The radar problem is different from fitting a dense image. In an image, supervision is dense on a regular grid. In the AMISR reconstruction problem, measurements are sparse in the spatial domain and constrained by radar beam geometry.

A high-frequency SIREN can fit sparse measurements while inventing structure between measured beams. That structure may be mathematically allowed but physically implausible.

Thus, the choice

$$\omega_0 = 5 \tag{140}$$

is best described as a low-frequency spectral prior.
It says:

prefer smoother large-scale plasma morphology unless the data and loss require otherwise. (141)

This does not freeze the final frequency content. The trainable weights can still change. The model can still represent structure. But the training starts in a lower-frequency regime and is biased toward smoother solutions.

17 Why $\omega_0 = 1$ does not automatically find the same result

Theoretically, a network with

$$\omega_0 = 1 \tag{142}$$

can represent the same functions by learning larger weights.
For example,

$$\sin(5wx) = \sin((5w)x). \tag{143}$$

However, optimization is not a global search over all equivalent functions. ADAM follows a path from the initialized weights.
The statements

$$\text{the model can represent the function} \tag{144}$$

and

$$\text{the optimizer will find that function} \tag{145}$$

are not equivalent.
This distinction is essential.
In a nonconvex neural-network optimization problem, the initialization and scaling affect:

- the initial function represented by the network,
- the initial gradient magnitudes,
- the rate at which high-frequency modes are learned,
- the stability of higher-order derivative losses,
- the basin or region of parameter space explored by training.

Therefore, $\omega_0 = 5$ is not used because $\omega_0 = 1$ cannot represent the result. It is used because $\omega_0 = 5$ produces a better optimization and inductive-bias regime for this problem.

18 Interaction between ω_0 and curvature regularization

The current loss contains

$$L_{xy} = \mathbb{E}\left[f_{\tilde{x}\tilde{x}}^2 + 2f_{\tilde{x}\tilde{y}}^2 + f_{\tilde{y}\tilde{y}}^2\right], \tag{146}$$

and

$$L_t = \mathbb{E}\left[f_{\tilde{t}\tilde{t}}^2\right]. \tag{147}$$

Since second derivatives of sine layers introduce factors of ω_0^2, the squared curvature terms can scale approximately like ω_0^4 in simple cases.

Therefore, ω_0 and curvature regularization are not independent. A higher ω_0 tends to make oscillatory curvature easier to create, but also more visible to the curvature penalty. A lower ω_0 makes the initial network smoother and reduces the tendency to create beam-to-beam oscillations.

The final behavior depends on the combination of:

$$\omega_0, \qquad W_\ell, \qquad L_{\text{data}}, \qquad \lambda_{xy}L_{xy}, \qquad \lambda_t L_t, \qquad \text{coordinate normalization.} \tag{148}$$

19 Horizontal Hessian penalty versus Laplacian penalty

The current spatial regularization is a horizontal Hessian penalty, not merely a Laplacian penalty.

For a function

$$f = f(\tilde{x}, \tilde{y}, \tilde{t}), \tag{149}$$

the horizontal Hessian is

$$H_{xy}f = \begin{bmatrix} f_{\tilde{x}\tilde{x}} & f_{\tilde{x}\tilde{y}} \\ f_{\tilde{y}\tilde{x}} & f_{\tilde{y}\tilde{y}} \end{bmatrix}. \tag{150}$$

Assuming sufficient smoothness,

$$f_{\tilde{x}\tilde{y}} = f_{\tilde{y}\tilde{x}}. \tag{151}$$

The squared Frobenius norm is

$$\|H_{xy}f\|_F^2 = f_{\tilde{x}\tilde{x}}^2 + 2f_{\tilde{x}\tilde{y}}^2 + f_{\tilde{y}\tilde{y}}^2. \tag{152}$$

The horizontal Laplacian is only

$$\nabla_{xy}^2 f = f_{\tilde{x}\tilde{x}} + f_{\tilde{y}\tilde{y}}. \tag{153}$$

A Laplacian-squared penalty would be

$$L_\Delta = \mathbb{E}\left[(f_{\tilde{x}\tilde{x}} + f_{\tilde{y}\tilde{y}})^2\right]. \tag{154}$$

This is not the same as the full Hessian penalty. The Hessian penalty includes mixed curvature through $f_{\tilde{x}\tilde{y}}$ and penalizes curvature in all horizontal directions.

20 Temporal curvature penalty

The temporal curvature penalty is

$$L_t = \mathbb{E}\left[f_{\tilde{t}\tilde{t}}^2\right]. \tag{155}$$

It is not a temporal-gradient penalty.
A temporal-gradient penalty would penalize

$$f_{\tilde{t}}^2. \tag{156}$$

That would discourage time evolution itself. In contrast, the second-derivative penalty discourages rapid bending in time.

For example,

$$f(\tilde{x}, \tilde{y}, \tilde{t}) = a(\tilde{x}, \tilde{y})\tilde{t} + b(\tilde{x}, \tilde{y}) \tag{157}$$

has

$$f_{\tilde{t}\tilde{t}} = 0. \tag{158}$$

Thus, a monotonic linear time evolution is allowed.

21 Automatic differentiation in training

The derivative losses are computed using automatic differentiation with respect to input coordinates. In the code, collocation coordinates are detached, cloned, and then marked with

```
requires_grad_(True)
```

The model output is

$$\hat{u} = f_\theta(\boldsymbol{r}_c), \tag{159}$$

where $\boldsymbol{r}_c$ is a collocation point.
Then automatic differentiation computes

$$\nabla_{\boldsymbol{r}} f_\theta = \begin{bmatrix} f_{\tilde{x}} \\ f_{\tilde{y}} \\ f_{\tilde{t}} \end{bmatrix}. \tag{160}$$

Second derivatives are then obtained by differentiating components of the gradient:

$$f_{\tilde{x}\tilde{x}} = \frac{\partial}{\partial \tilde{x}} \left(f_{\tilde{x}}\right), \tag{161}$$

$$f_{\tilde{x}\tilde{y}} = \frac{\partial}{\partial \tilde{y}} \left(f_{\tilde{x}}\right), \tag{162}$$

$$f_{\tilde{y}\tilde{y}} = \frac{\partial}{\partial \tilde{y}} \left(f_{\tilde{y}}\right), \tag{163}$$

$$f_{\tilde{t}\tilde{t}} = \frac{\partial}{\partial \tilde{t}} \left(f_{\tilde{t}}\right). \tag{164}$$

This is aligned with the SIREN paper's use of derivatives of the neural field with respect to input coordinates.

22 How the current implementation aligns with SIREN

The current radar implementation is SIREN-aligned in the following sense.

22.1 Coordinate input

The model input is

$$[\tilde{x}, \tilde{y}, \tilde{t}]. \tag{165}$$

This is a coordinate-based neural field, as in the original SIREN framework.

22.2 Sine activation

The activation is

$$\sin(w_0 z), \tag{166}$$

where

$$z = Wh + b. \tag{167}$$

The code makes w_0 configurable through

$$\texttt{first\_omega\_0} \tag{168}$$

and

$$\texttt{hidden\_omega\_0}. \tag{169}$$

22.3 SIREN-style initialization

The first layer uses

$$W_1 \sim \mathcal{U}\left(-\frac{1}{n_{\text{in}}}, \frac{1}{n_{\text{in}}}\right). \tag{170}$$

The hidden layers use

$$W_\ell \sim \mathcal{U}\left(-\frac{\sqrt{6/n_{\text{in}}}}{\omega_{\text{hidden}}}, \frac{\sqrt{6/n_{\text{in}}}}{\omega_{\text{hidden}}}\right). \tag{171}$$

This is the key SIREN initialization idea generalized from a hard-coded $\omega_0 = 30$ to a configurable ω_0.

22.4 Linear output

The final layer is linear:

$$f_\theta(\boldsymbol{r}) = W_{\text{out}} h_L + \boldsymbol{b}_{\text{out}}. \tag{172}$$

This is appropriate for scalar regression.

22.5 Derivative losses

The model uses automatic differentiation to compute spatial and temporal second derivatives at collocation points. This is consistent with the SIREN motivation, which emphasizes that the network's derivatives should be well-behaved.

23 Important caveats

23.1 The output flag

The model has an argument called

$$\texttt{outermost\_linear}. \tag{173}$$

However, in the current implementation, the output layer is always linear. This is fine for the present use case, but the argument is currently misleading unless a branch is added to apply an output activation when `outermost_linear=False`.

23.2 Derivative units

The curvature losses are currently computed in normalized coordinate units. Therefore,

$$L_{xy} \tag{174}$$

is not directly a physical curvature in units such as km^{-2} unless the chain-rule scale factors are included.

This is not necessarily wrong. It simply means the regularizer is a normalized-coordinate prior.

23.3 Collocation time sampling

The current collocation pool uses real radar time records. This means the temporal curvature penalty is evaluated at measured time planes.

A possible future improvement is to sample continuous normalized times,

$$\tilde{t}_c \sim \mathcal{U}(-1, 1), \tag{175}$$

so that temporal curvature is also controlled between radar records.

24 Recommended explanation to a neural-network expert

A concise but technically accurate explanation is:

> The SIREN frequency-scale parameter ω_0 is not treated as a trainable model parameter. The trainable parameters are the linear-layer weights and biases. Mathematically, ω_0 could be absorbed into the weights, so it is not required for expressivity. Its role is instead to control initialization, optimization conditioning, gradient scales, higher-order derivative scales, and the spectral prior of the coordinate network. In the sparse AMISR reconstruction problem, $\omega_0 = 5$ is used as a low-frequency architectural prior: it biases the model toward smoother plasma morphology while still allowing the trainable weights to adapt. Making ω_0 trainable is possible, but it introduces a redundant non-identifiable scale parameter and may optimize for lower pointwise training error rather than physically plausible interpolation.

25 Recommended explanation of the higher-order derivative issue

A concise derivative-focused explanation is:

> For a neuron $f(x) = \sin(\omega_0(wx + b))$, the first derivative scales as $\omega_0 w$, the second derivative scales as $(\omega_0 w)^2$, and the k-th derivative scales as $(\omega_0 w)^k$. Therefore ω_0 strongly affects derivative magnitudes. Since our radar loss penalizes second derivatives, the squared curvature terms can scale approximately as ω_0^4 in simple cases. This means ω_0 is not only a visual frequency knob. It directly affects the magnitude and conditioning of the derivative losses used during training.

26 Conclusion

The current radar implementation is a valid SIREN-style implicit neural representation. It differs from the original image examples mainly because it uses lower frequency scale, sparse radar data, temporal windows, and curvature regularization.

The most important conceptual point is that ω_0 should not be described as a learned frequency. It is a fixed activation-scale hyperparameter. The weights and biases learn the function. The ω_0 value controls the initial spectral regime and strongly affects gradients and higher-order derivatives.

Thus, the correct interpretation of $\omega_0 = 5$ is:

$$\boxed{\omega_0 = 5 \text{ is a low-frequency SIREN prior for sparse radar-field reconstruction.}} \tag{176}$$

It is not a claim that the true plasma frequency is 5, and it is not a claim that the network cannot learn other frequency content. It is a practical and defensible choice that affects initialization, optimization, derivative behavior, and interpolation bias.

References

[1] Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. *Implicit Neural Representations with Periodic Activation Functions*. NeurIPS, 2020.

[2] Vincent Sitzmann. *SIREN GitHub repository*. https://github.com/vsitzmann/siren

[3] SIREN project website. https://www.vincentsitzmann.com/siren/